Universidad Autónoma de Tamaulipas
Facultad de Ingeniería Tampico

VERDAD, BELLEZA, PROBIDAD

INGENIERÍA TAMPICO

ASIGNATURA
Programación de Interfaces y Puertos
6°. Semestre – Grupo "I"
2025 -1

TRABAJO
Programas en Clase y Ejercicios

UNIDAD
1 - Modelos de interacción computacional

Docente: Dr. García Ruiz Alejandro H.

Integrante del Equipo	Nivel de Participación
Vicente Bautista Roberto Nicolas	33.33%
Rosales Pereles Denisse Ariadna	33.33%
Morales Urrutia Javier Antonio	33.33%
Total:	100%

Aniversario 75 UAT 1950-2025

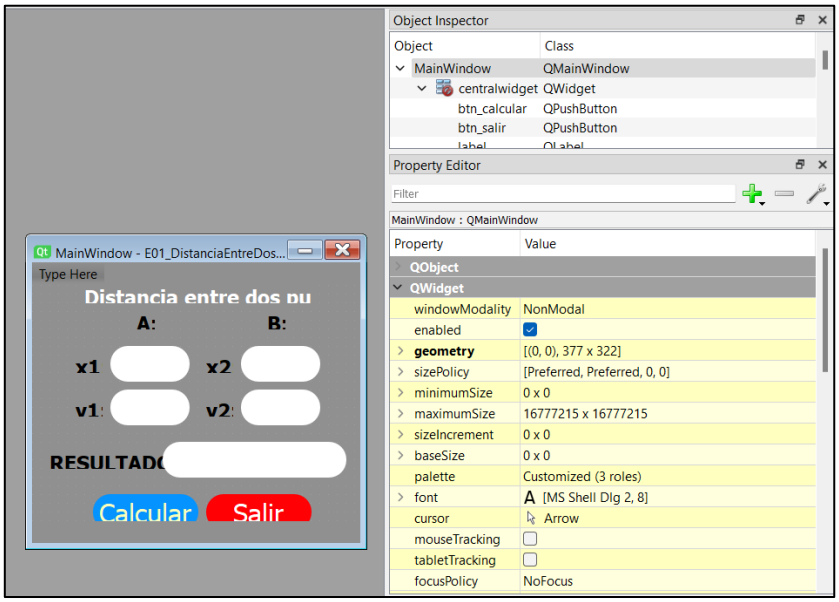
Índice

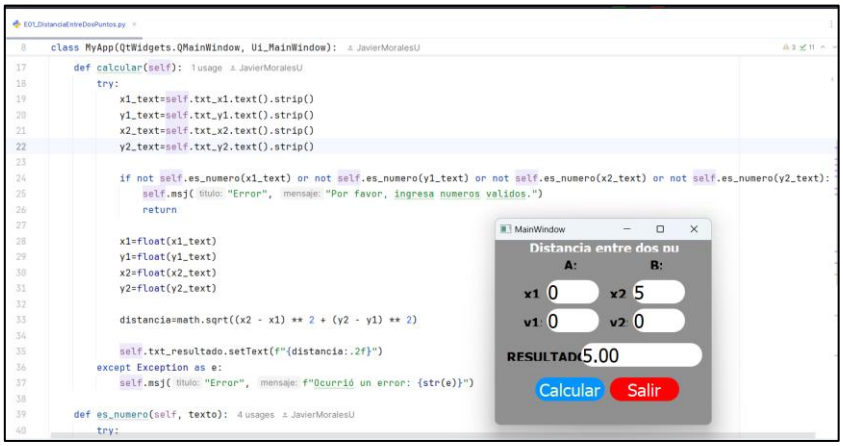
Índice 1
Repositorio(s) 2
E01_Distancia entre dos puntos 2
E02_Par o impar 3
E03_Area de un cuadrado 4
E04_Area de un pentágono 5
E05_Edad 6
E06_Tabla de multiplicar 7
E07_Caracteres de una cadena 8
E08_Formato 24hr 9
E09_Factorial 10
P00_Introduccion 11
P 01_Ejemplo con un botón 12
P 02_Mensaje emergente básico 13
P 03_Caja de texto LineEdit 14
P 04_Suma de dos números 15
P 05_Suma de dos números, resultado en LineEdit 16
P 06_Suma de múltiples números 17
P 07_Suma de múltiples números, carga 18
P 08_Suma de múltiples números, lista de números 19

Repositorio(s)

Actividad	Repositorio
Ejercicios	https://github.com/JavierMoralesU/PIP/tree/master/Unidad%201/Ejercicios

E01_Distancia entre dos puntos

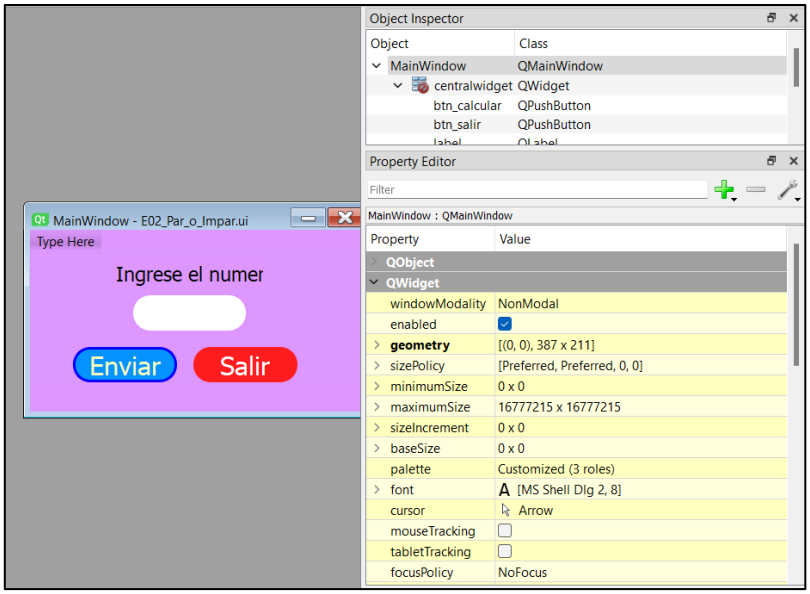


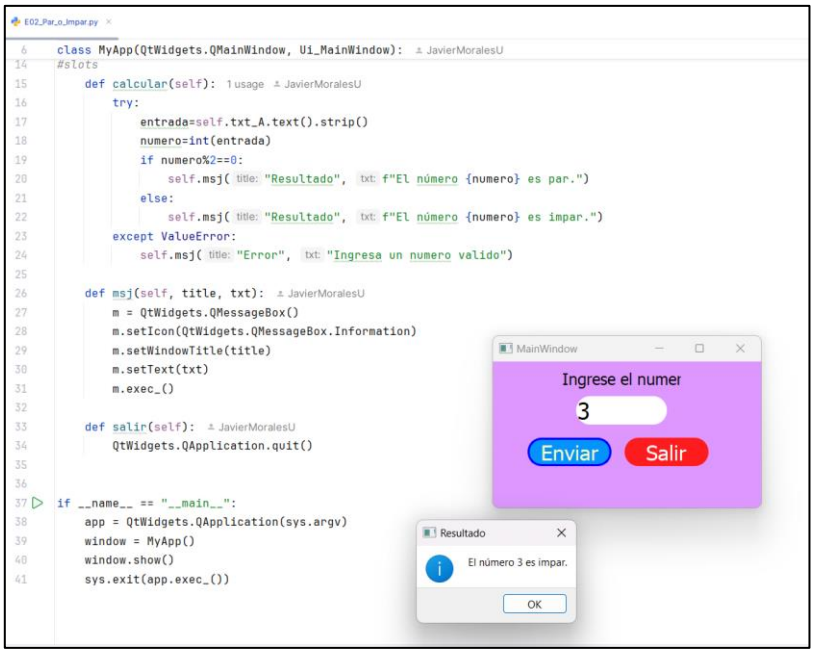


Se usan QLineEdit para que el usuario ingrese números que representan coordenadas, se usa un método es número para comprobar que los datos ingresados sean números que se puedan convertir a flotante. Una vez obtenido los números se usa la formula para calcular distancias y muestra el resultado con el metodo msj, que muestra un QMessageBox().

$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ -> distancia=math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)

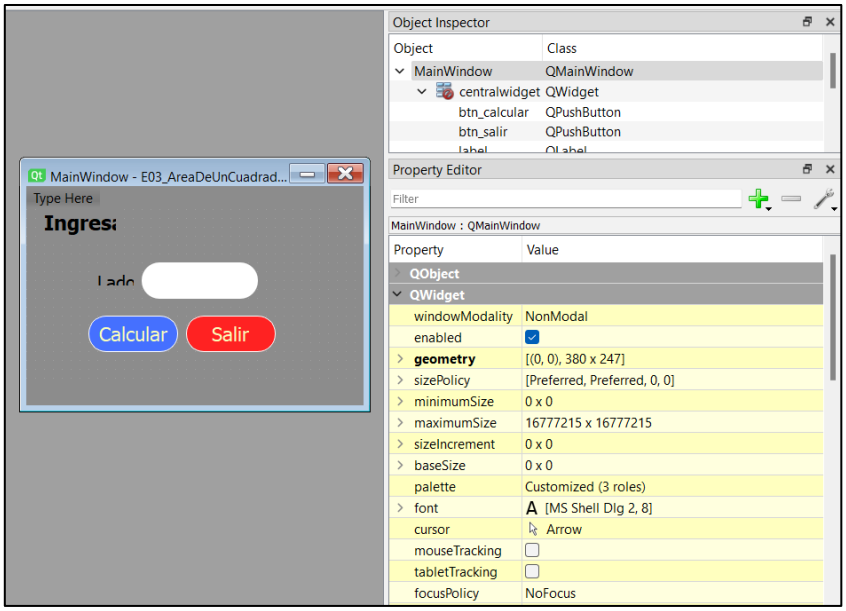
E02_Par o impar

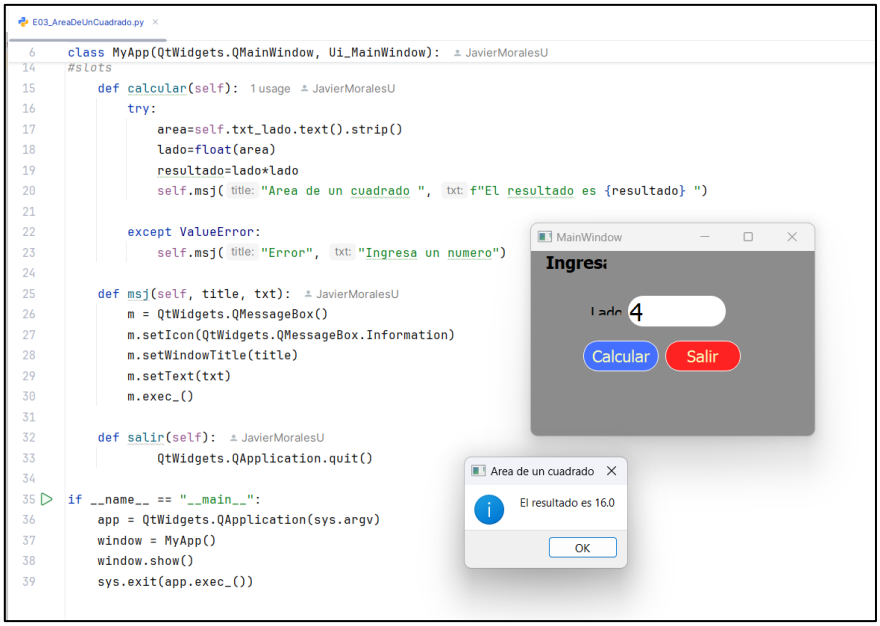




El programa obtiene un numero ingresado por el usuario dentro del QLineEdit, y evalúa si el numero es par o impar, para ello obteniendo el residuo de dividir el numero entre dos (%2) Y muestra el mensaje en una ventana emergente.

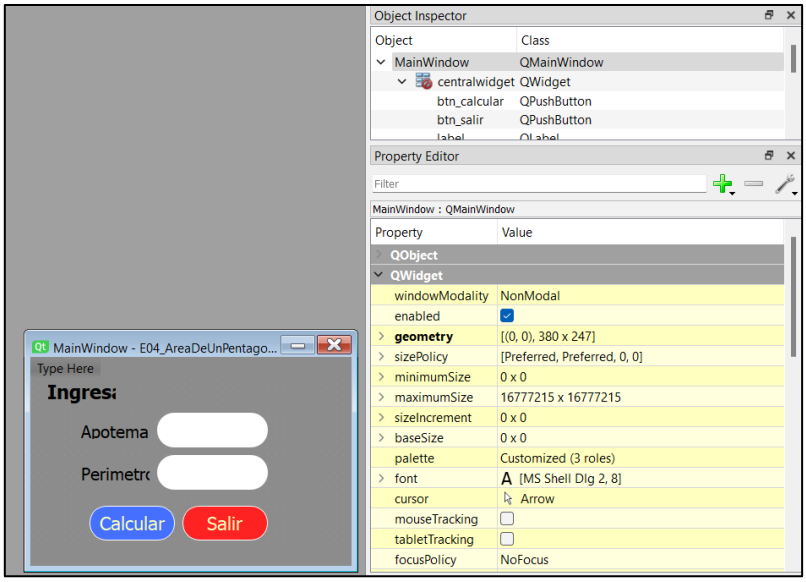
E03_Area de un cuadrado

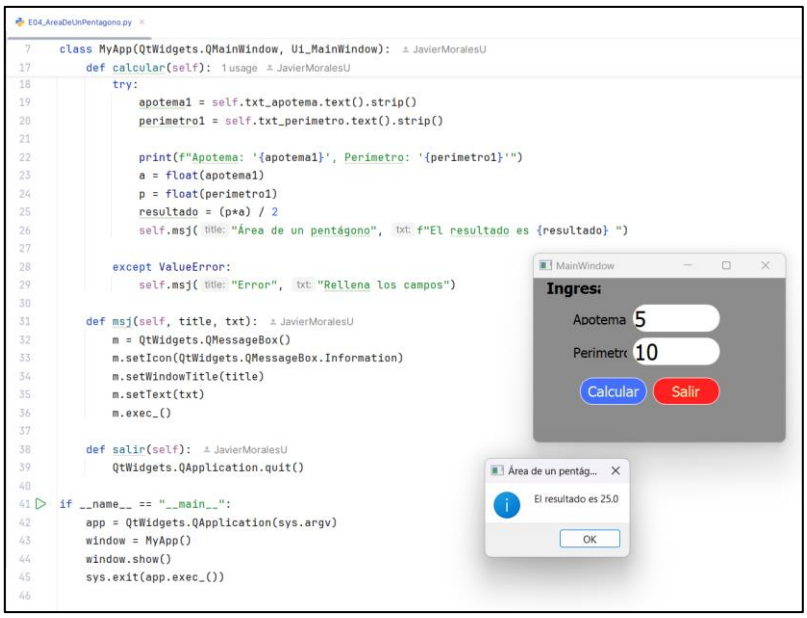




Multiplica el numero ingresado por si mismo para obtener el cuadrado que se muestra en una ventana emergente. Maneja excepciones ValueError para en caso de que se introduzca algo que no sea un numero.

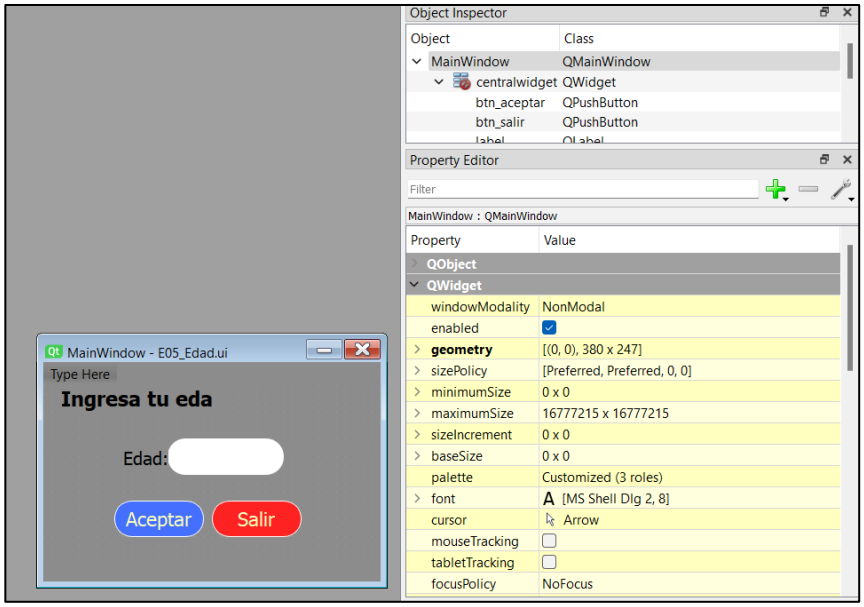
E04_Area de un pentágono

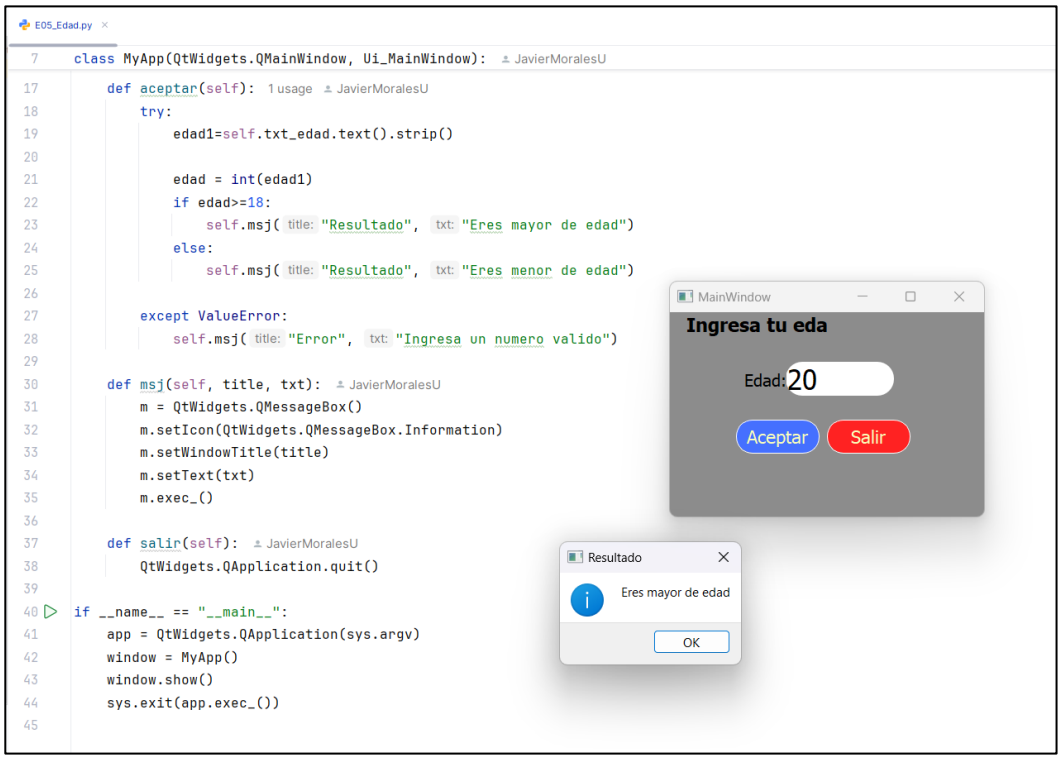




Utiliza la fórmula del área de un pentágono (perímetro * apotema / 2) y muestra el resultado a través de una ventana emergente. Utiliza un control de excepciones para el caso de que se ingrese algo diferente a números o que los campos se hallan dejado vacíos.

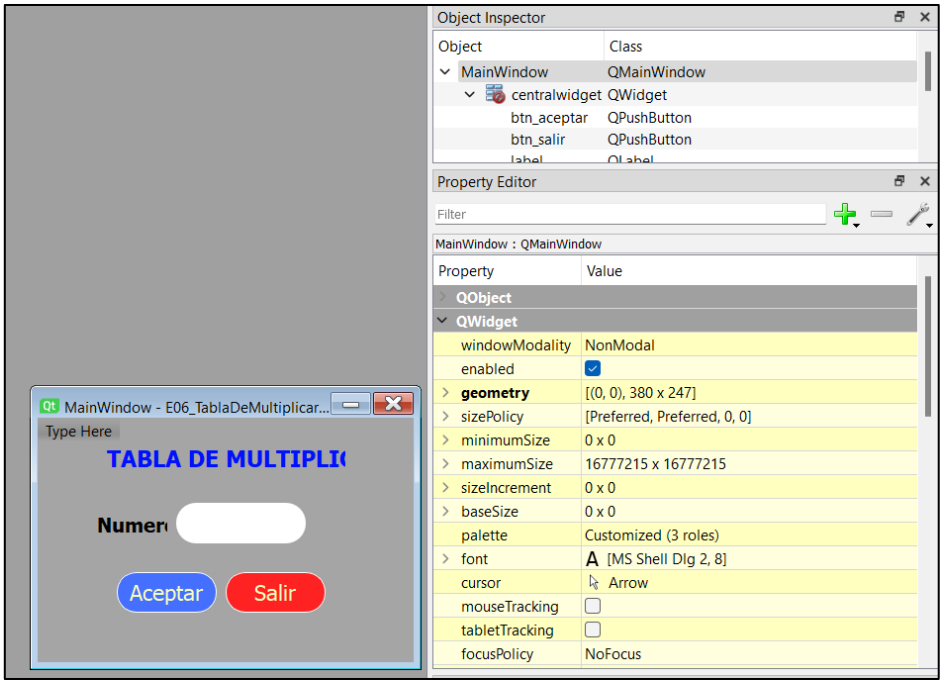
E05_Edad

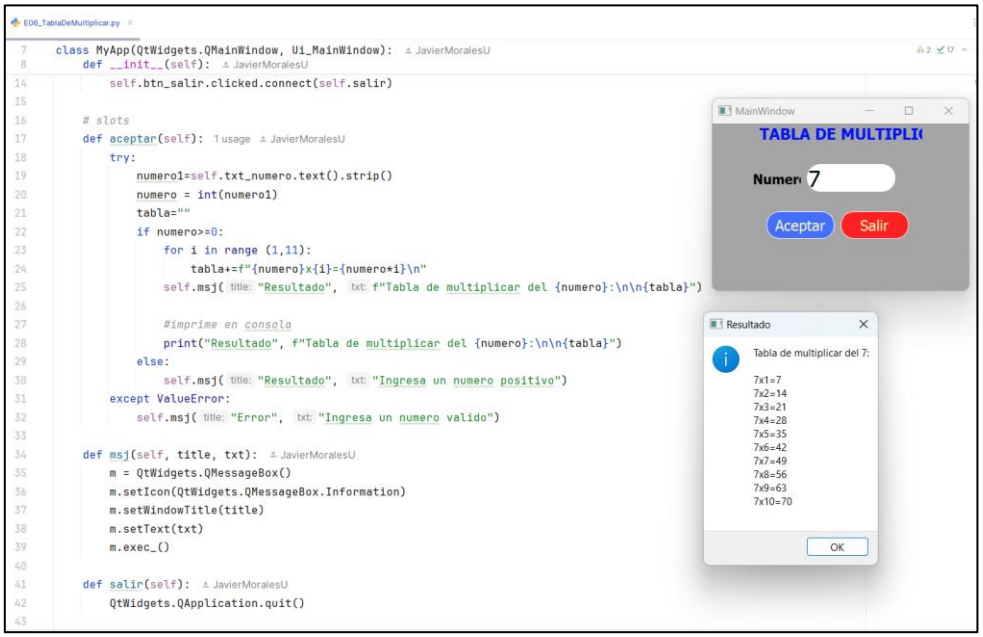




Se obtiene el numero ingresado por el usuario, y se evalúa si el número es igual o mayor a 18, en ese caso la ventana emergente dice que eres mayor de edad, si no, eres menor de edad.

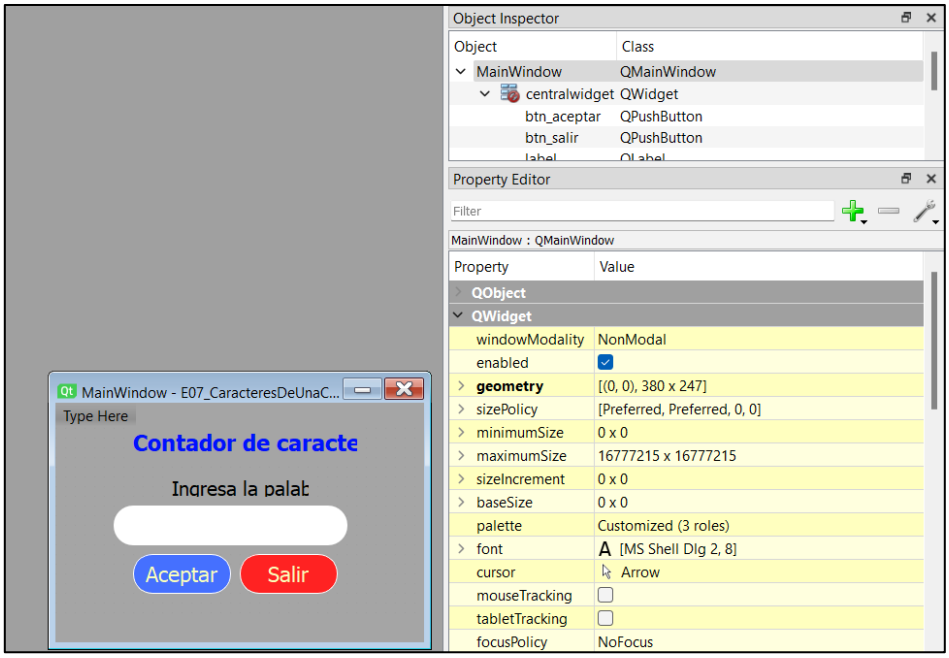
E06_Tabla de multiplicar

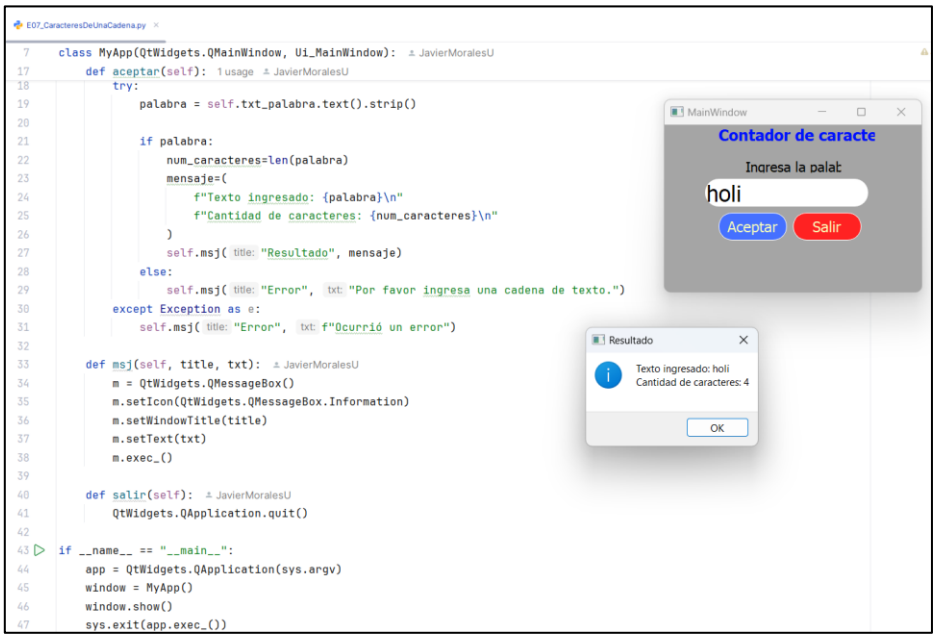




Muestra la tabla de multiplicar del número ingresado, concatenando las 10 operaciones en forma de texto dentro de una variable que después será mostrada en la ventana emergente.

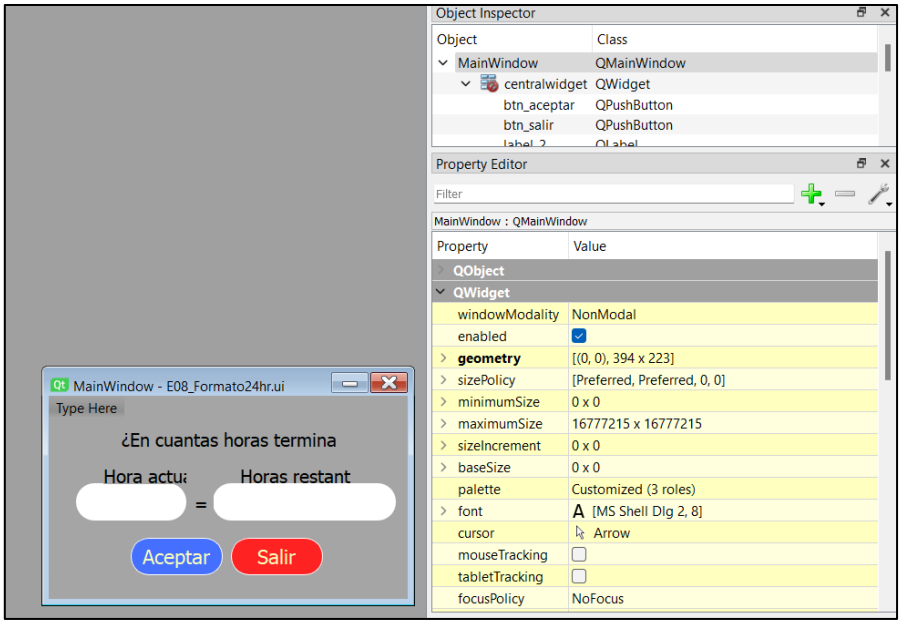
E07_Caracteres de una cadena

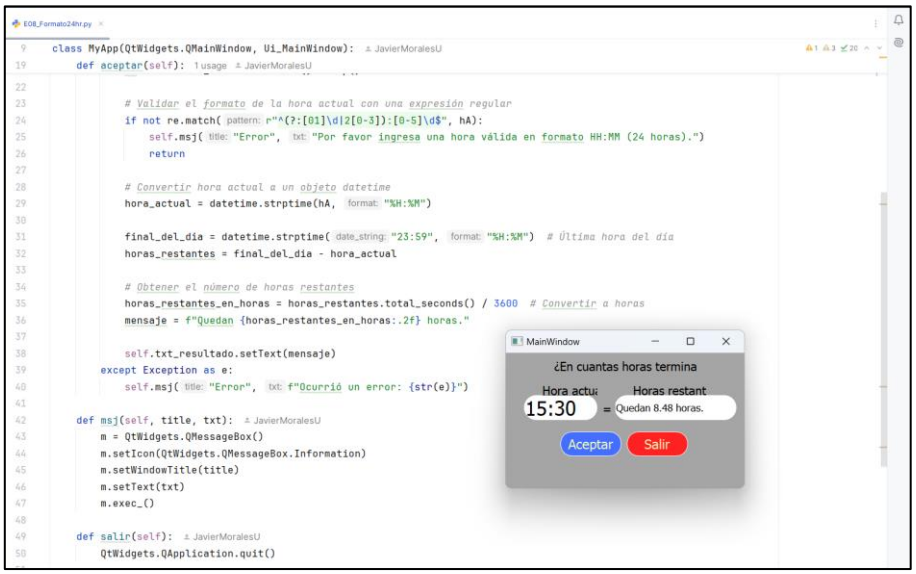




El código usa el método len para obtener la cantidad de caracteres que el usuario ingresa, y lo muestra en la ventana de resultado.

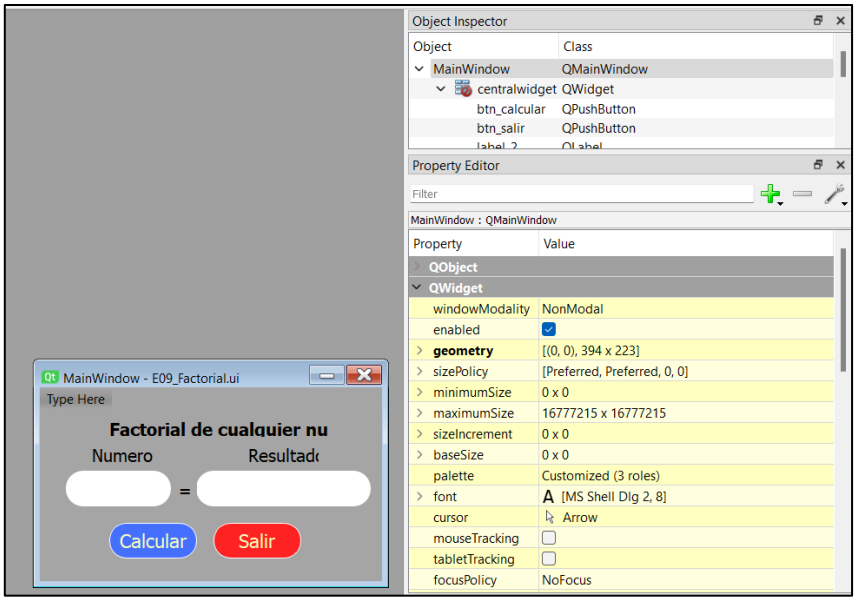
E08_Formato 24hr

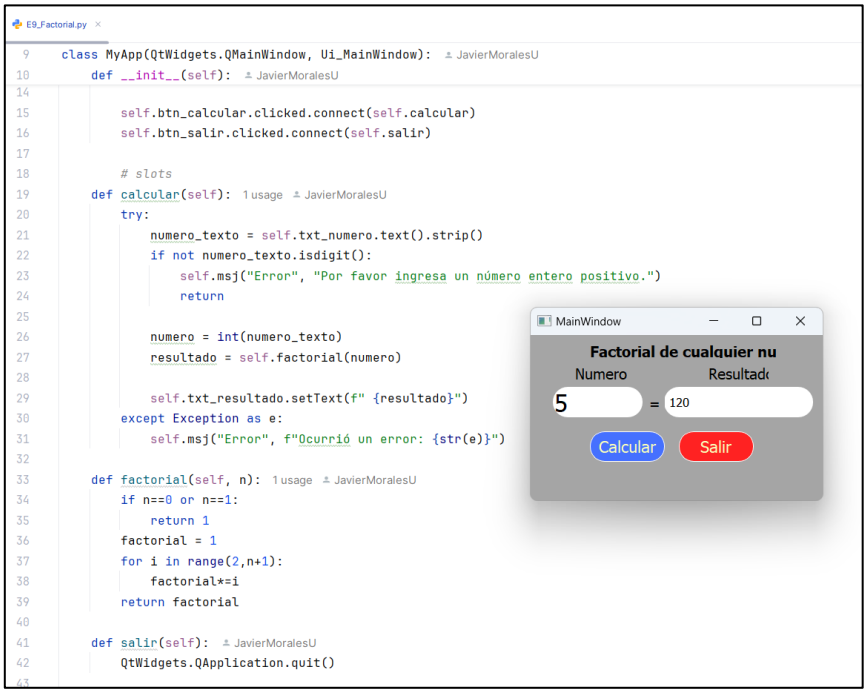




Se verifica que la hora ingresada por el usuario cumple con el formato de 24 horas, y procede a convertirla en un objeto de tipo datetime con su método .strptime, para poder restarlo a otro que representa 23:59, y este tiempo restante es obtenido y transformado a horas para después mostrar en el otro QLineEdit. Para ello se usó la librería de Python datetime que permite representar horas del día.

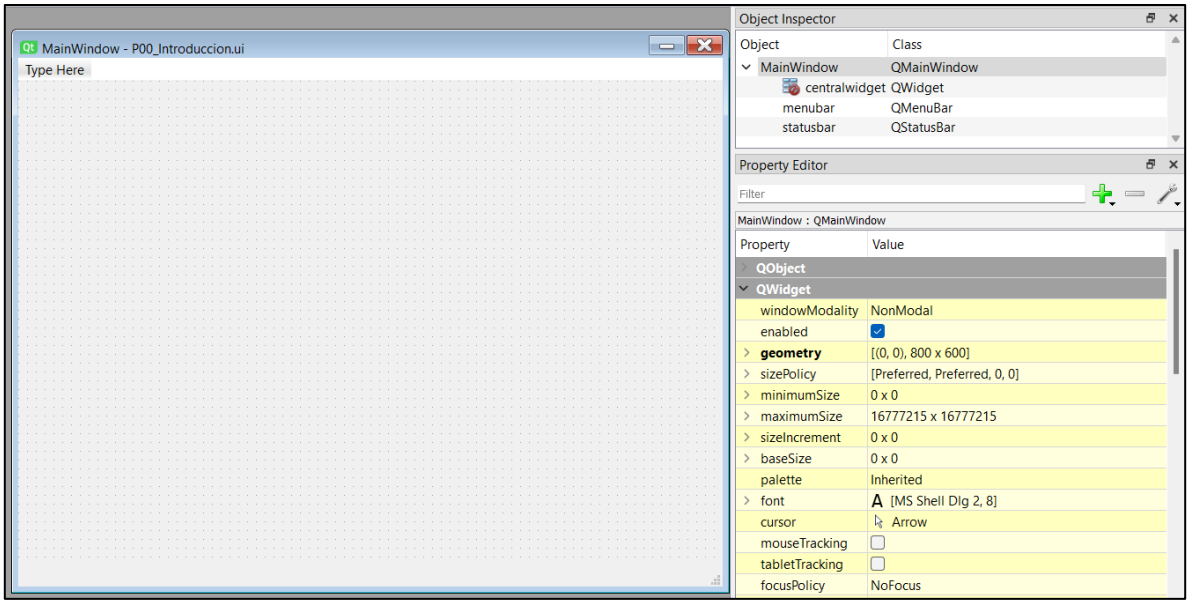
E09_Factorial

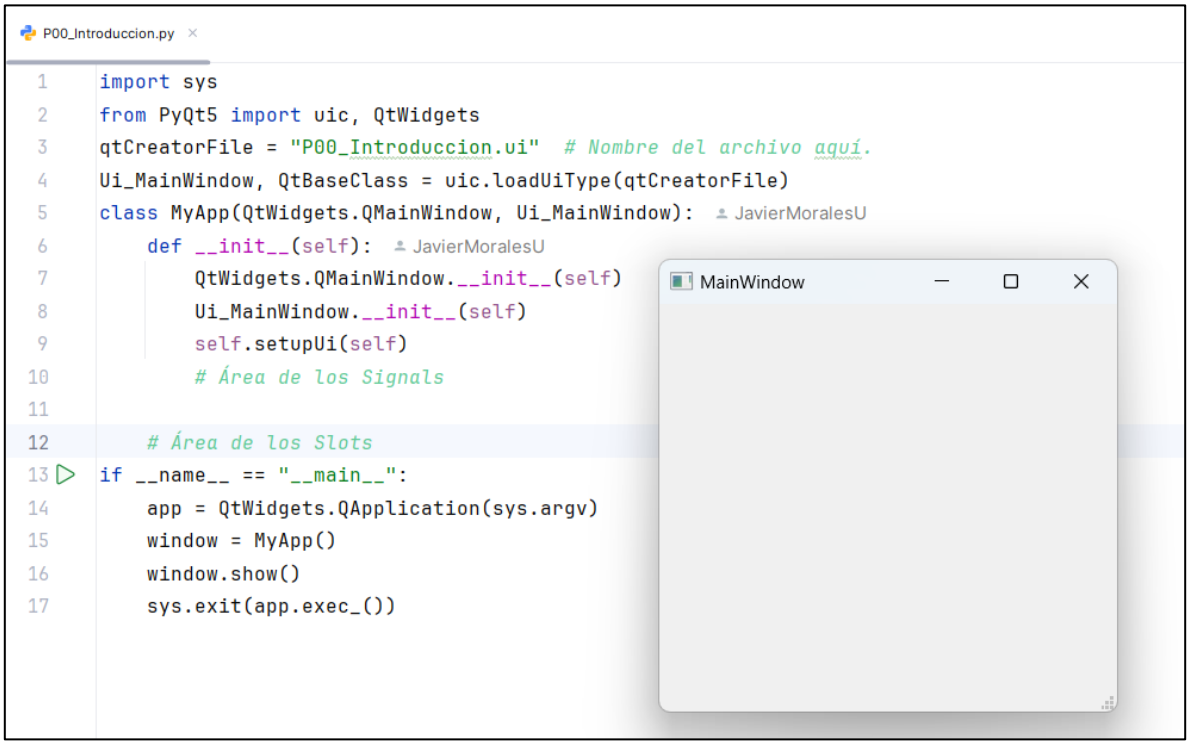




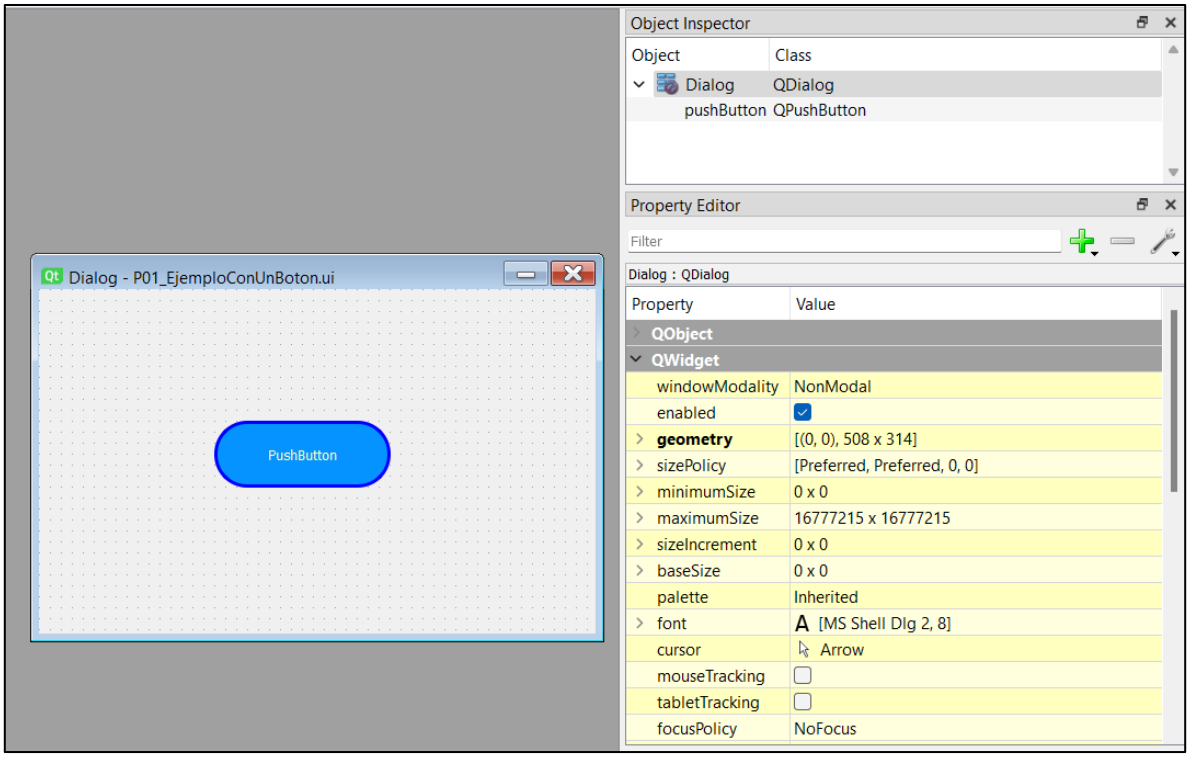
Se calcula el factorial del número ingresado (no aceptando negativos), para ello se usa el método factorial que se usa al oprimir el botón conectado. Se empieza con el 1 en la variable factorial y con ayuda de un for se va multiplicando por los números recorridos en este hasta llegar al propio numero ingresado, ejemplo : 1 * 2 * 3 * 4 * 5 = 120

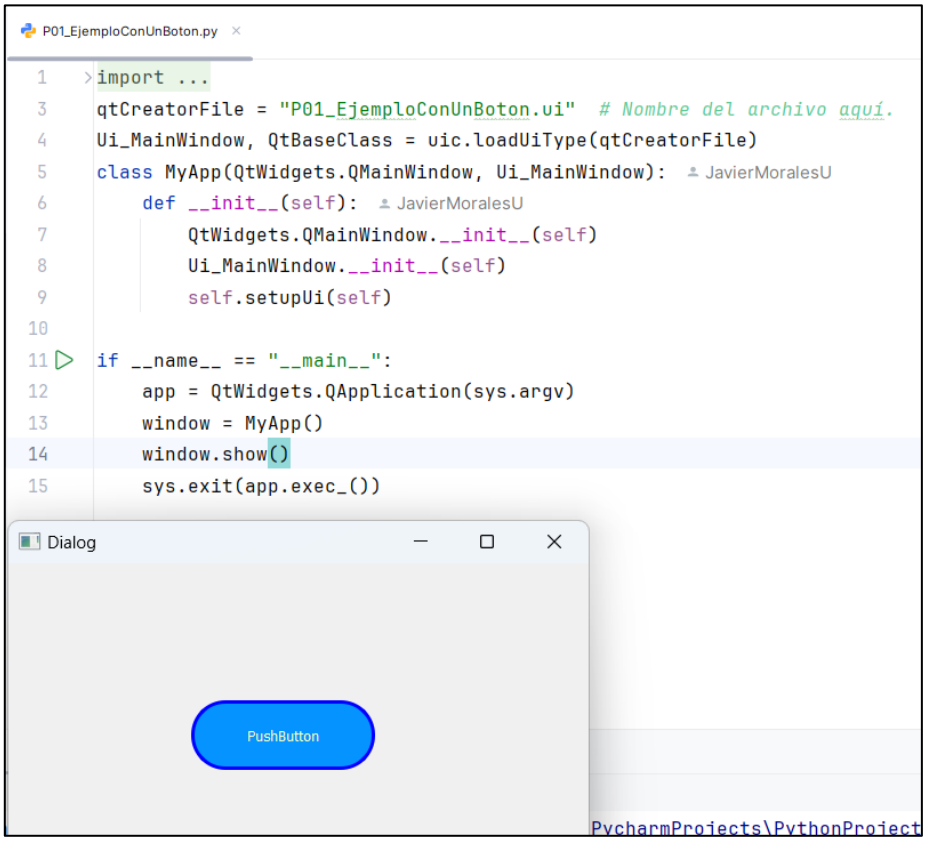
P00_Introduccion



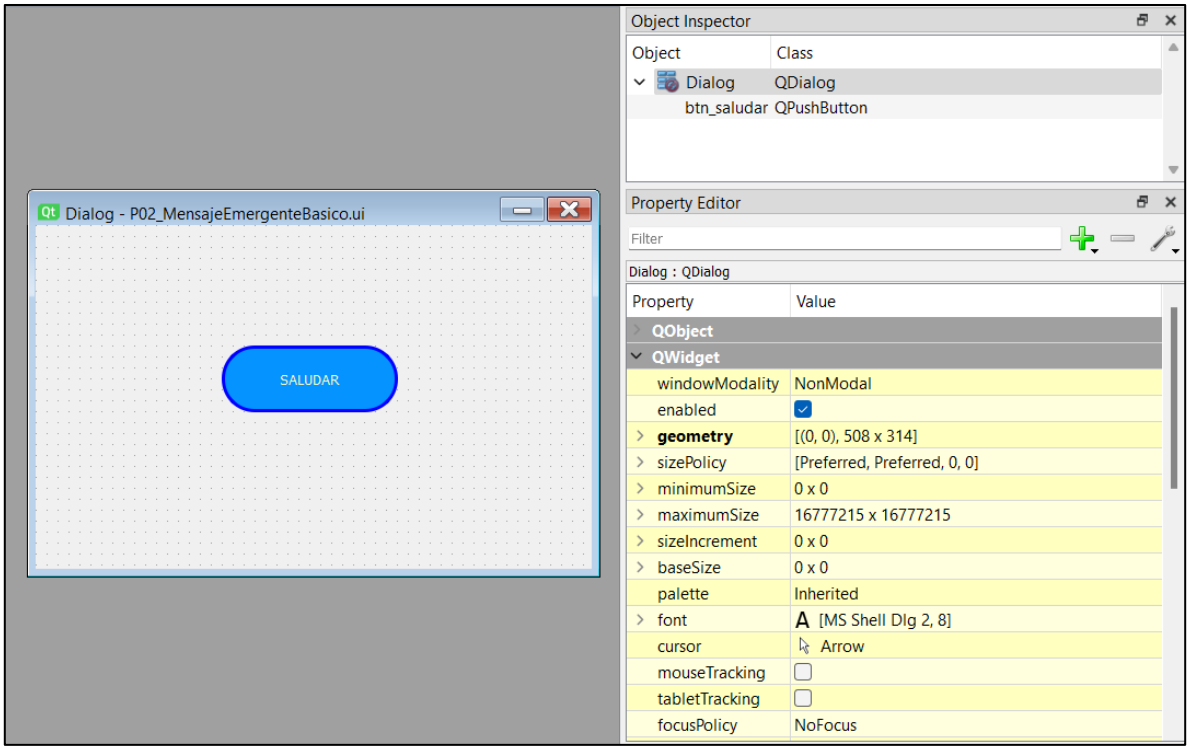


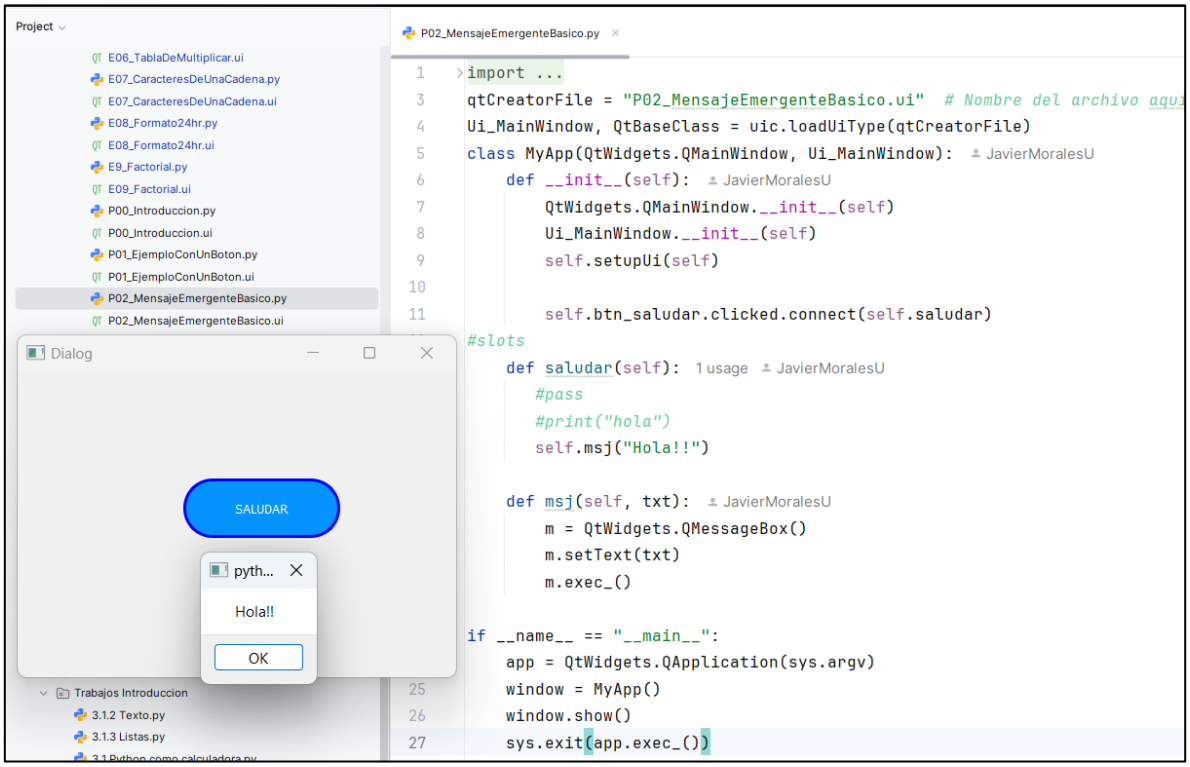
P 01_Ejemplo con un botón



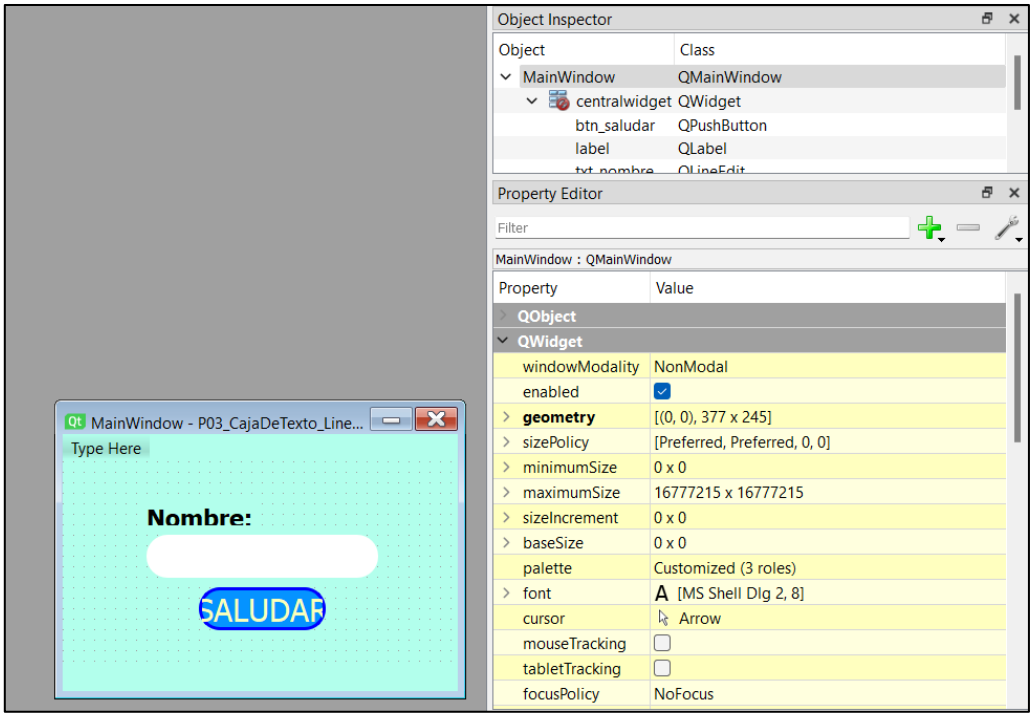


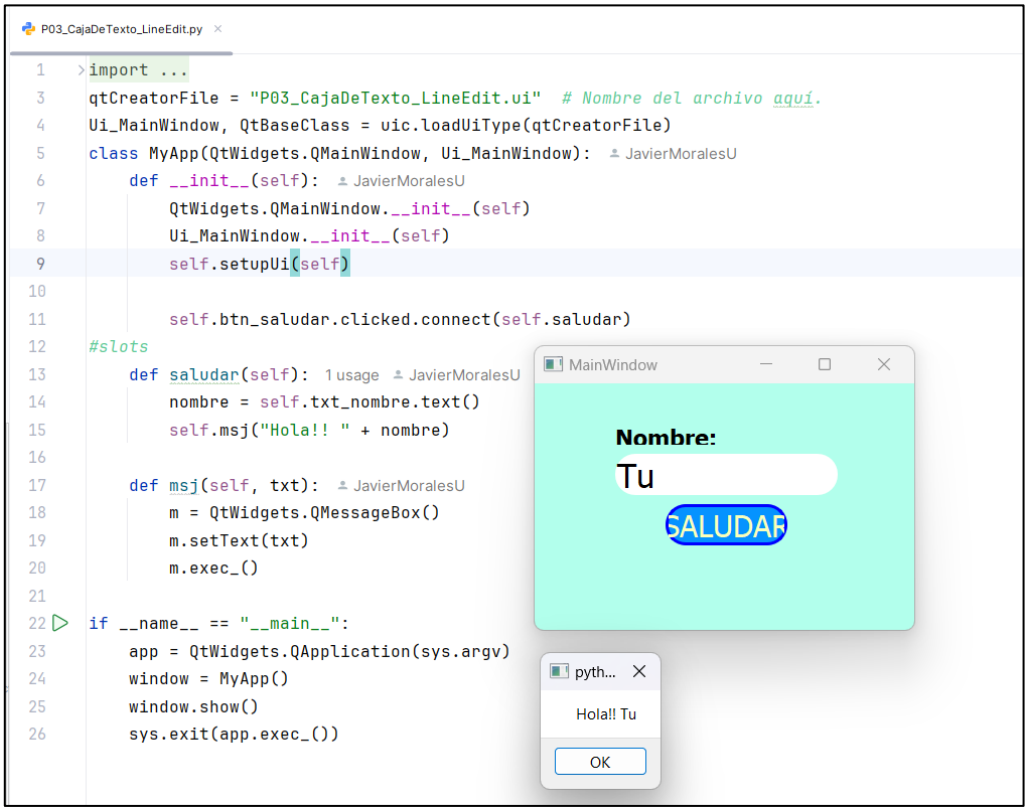
P 02_Mensaje emergente básico



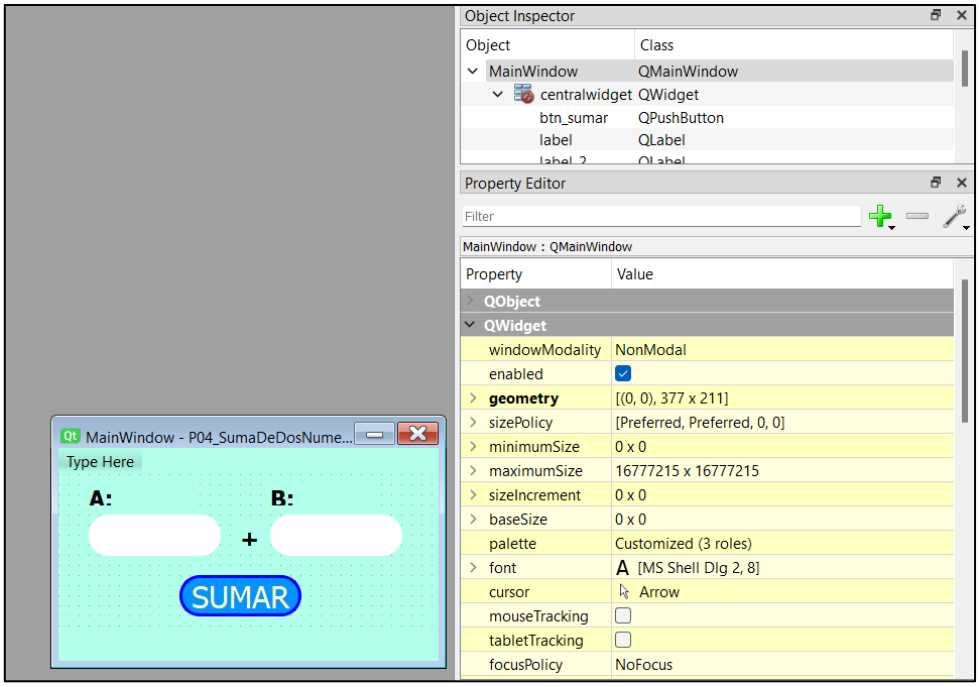


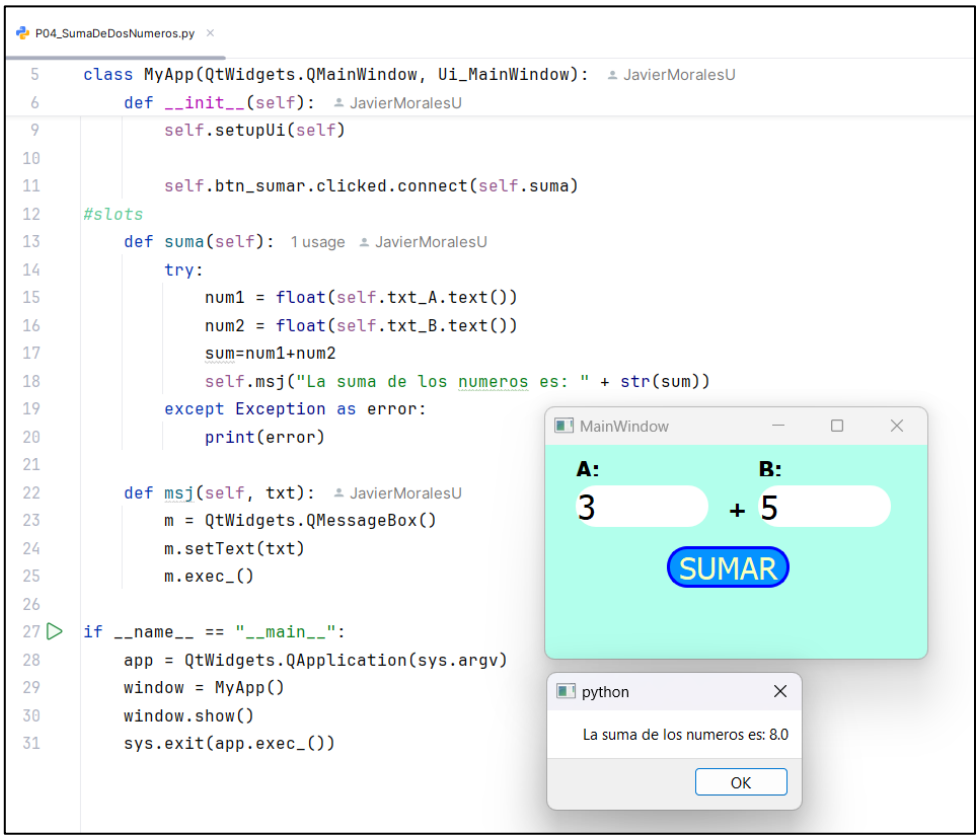
P 03_Caja de texto LineEdit



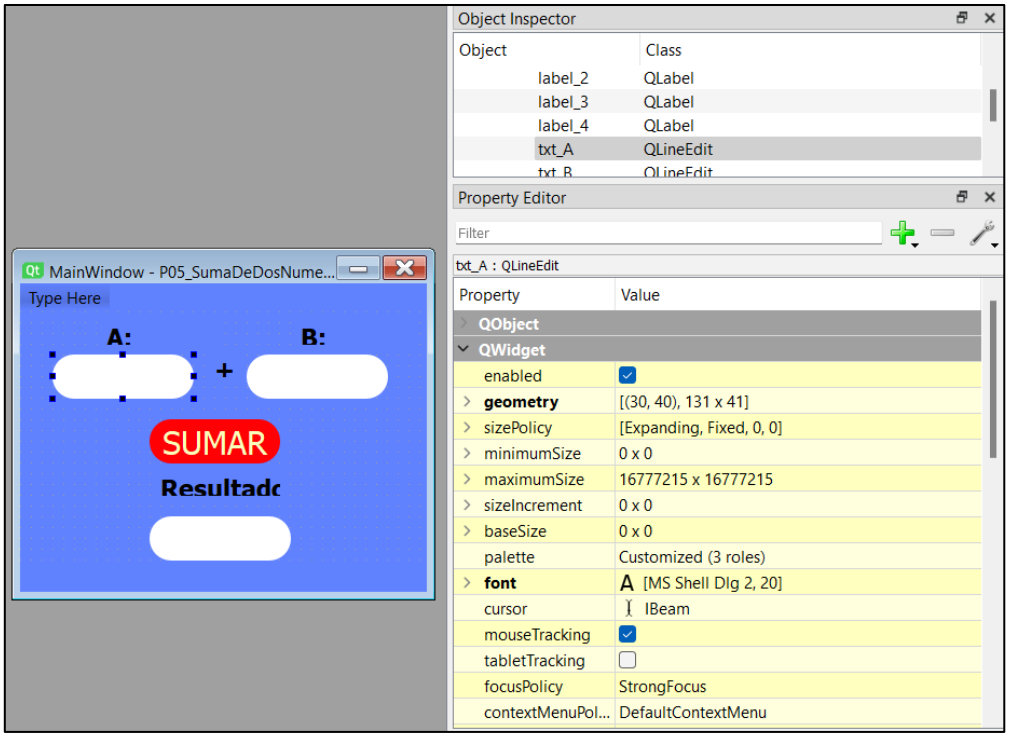


P 04_Suma de dos números



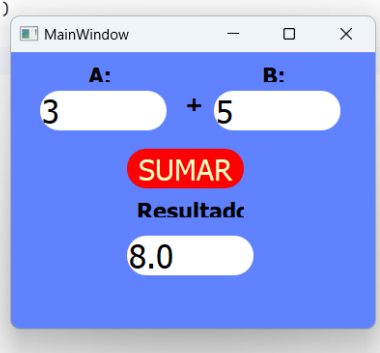


P 05_Suma de dos números, resultado en LineEdit

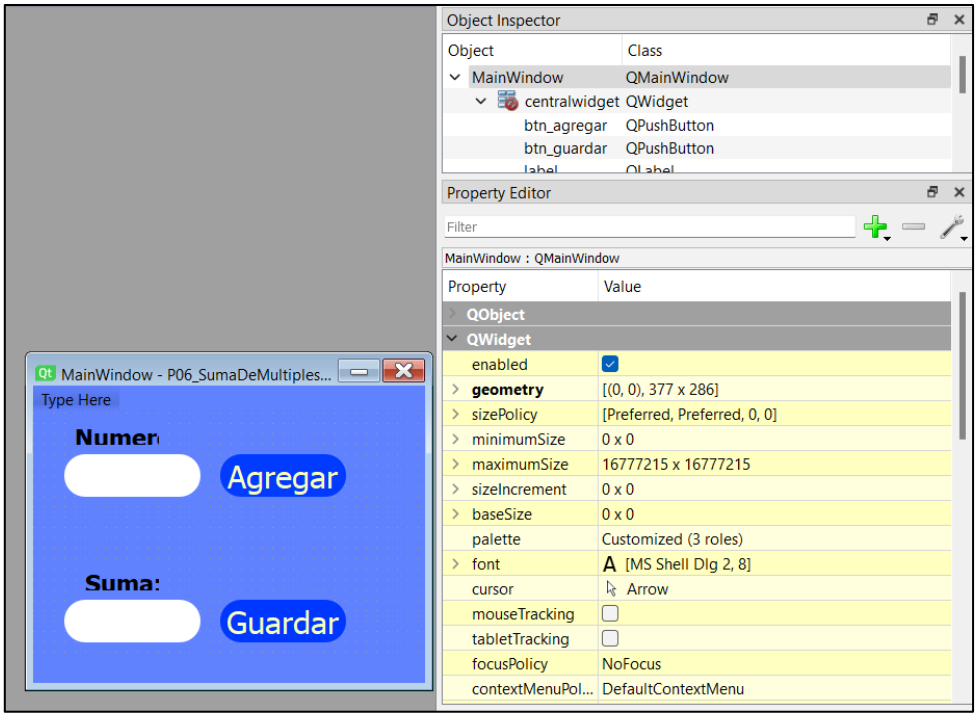


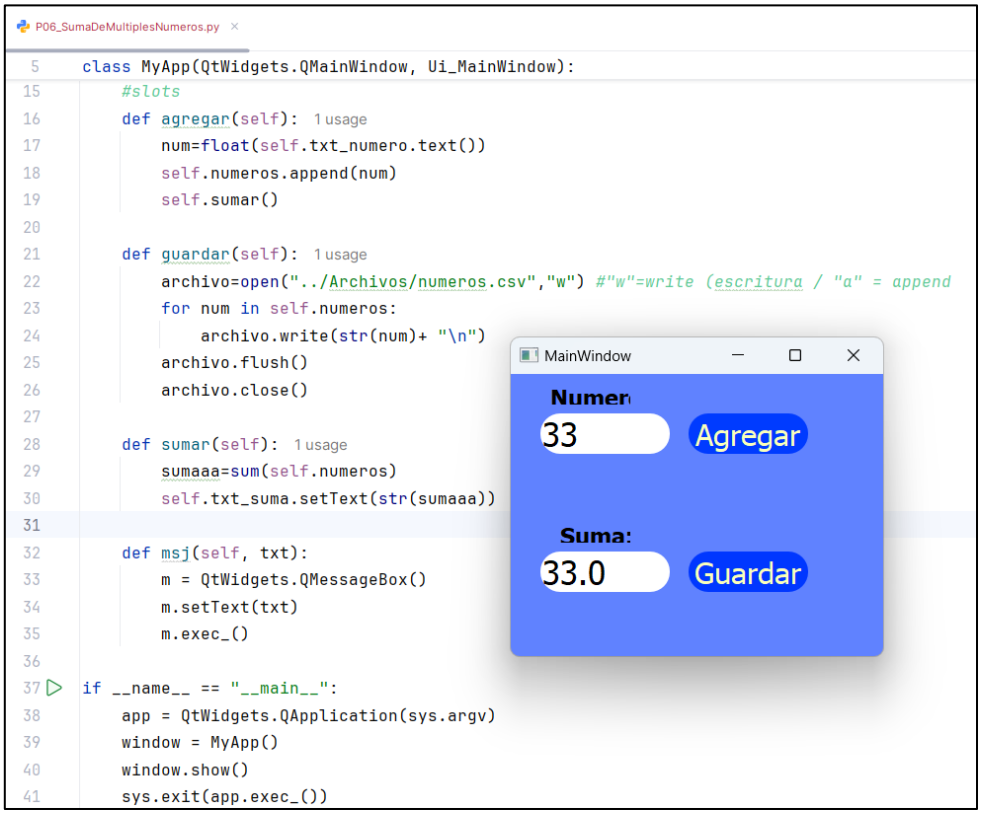
P05_SumaDeDosNumeros.py

```python
class MyApp(QtWidgets.QMainWindow, Ui_MainWindow):
    def __init__(self):

        self.btn_sumar.clicked.connect(self.suma)
#slots
    def suma(self):  1 usage
        try:
            num1 = float(self.txt_A.text())
            num2 = float(self.txt_B.text())
            sum=num1+num2
            #self.msj("La suma de los numeros es: " + str(sum))
            self.txt_resultado.setText(str(sum))
        except Exception as error:
            print(error)

    def msj(self, txt):
        m = QtWidgets.QMessageBox()
        m.setText(txt)
        m.exec_()

if __name__ == "__main__":
    app = QtWidgets.QApplication(sys.argv)
    window = MyApp()
    window.show()
    sys.exit(app.exec_())
```

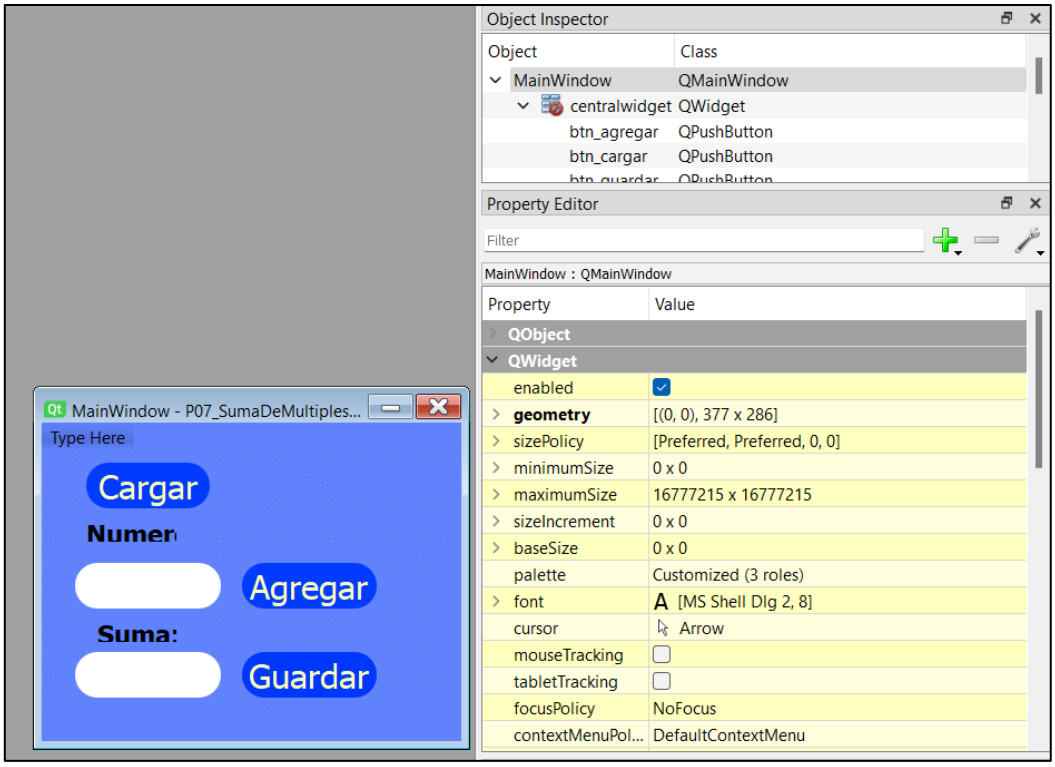


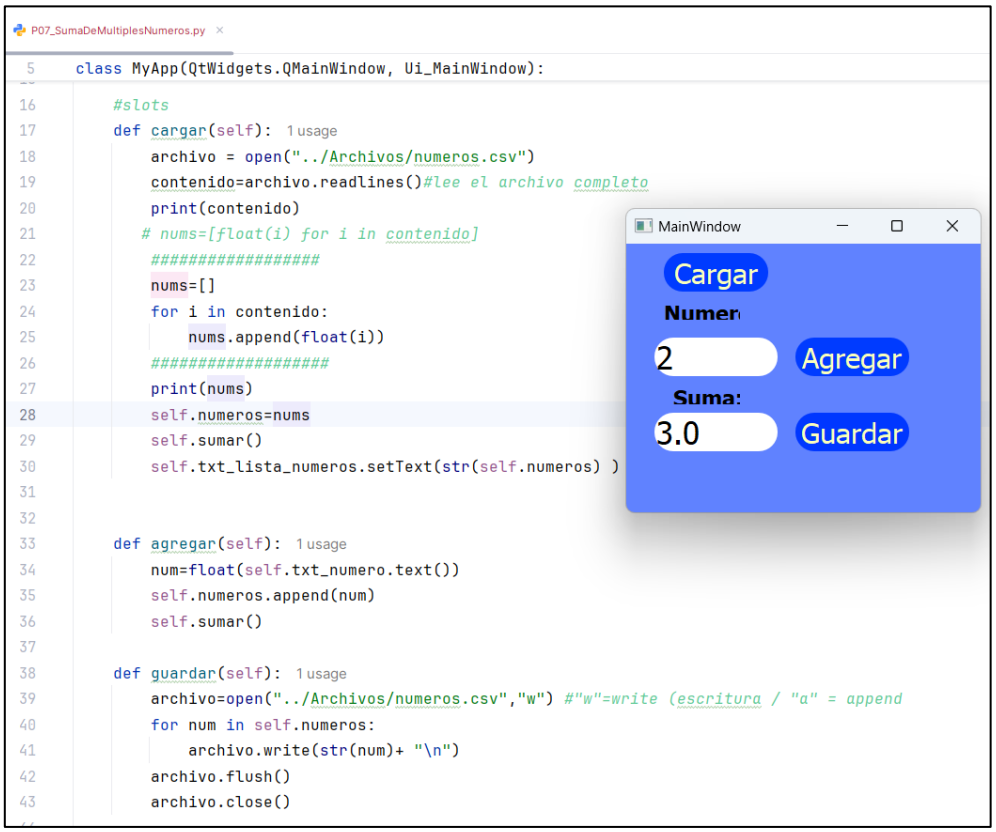
P 06_Suma de múltiples números



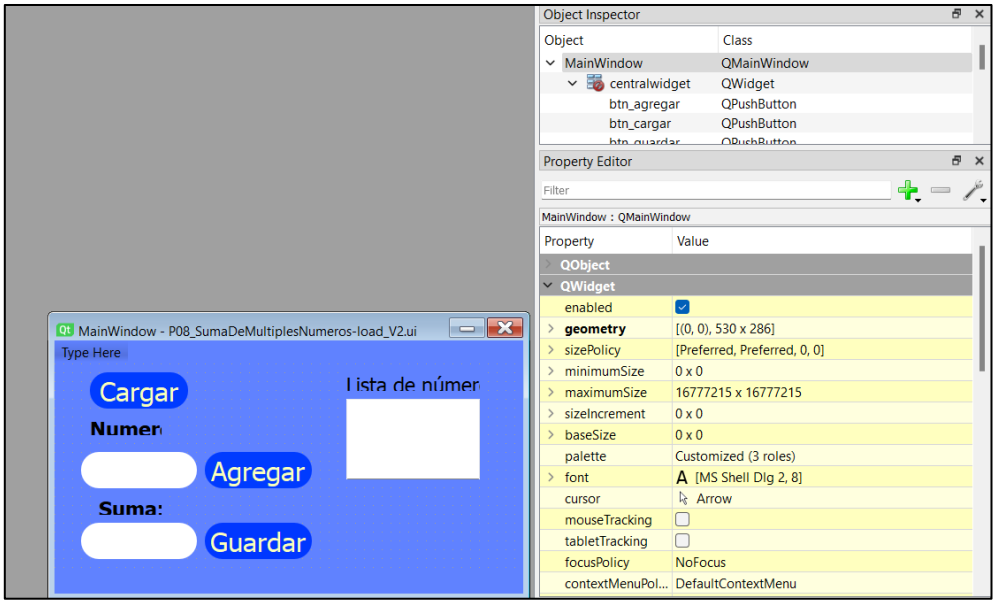


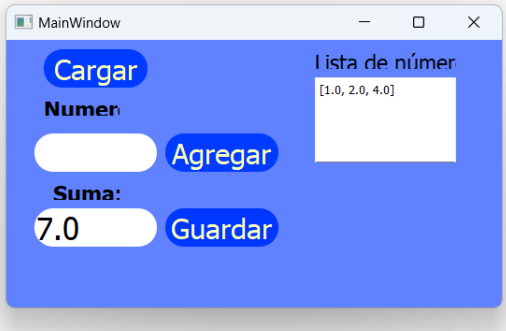
P 07_Suma de múltiples números, carga





P 08_Suma de múltiples números, lista de números



P08_SumaDeMultiplesNumeros-load_V2.py

```python
class MyApp(QtWidgets.QMainWindow, Ui_MainWindow):
    def __init__(self):
        self.ruta_archivo = os.path.join("..", "..", "Archivos", "numeros.csv")

        self.btn_agregar.clicked.connect(self.agregar)
        self.btn_guardar.clicked.connect(self.guardar)
        self.btn_cargar.clicked.connect(self.cargar)
        self.numeros=[] #lista vacia, que puede ser utilizada en cualquier parte de la

    #slots
    def cargar(self):  1 usage
        archivo = open(self.ruta_archivo)
        contenido=archivo.readlines()#lee el archivo completo
        print(contenido)
       # nums=[float(i) for i in contenido]
        ##################
        nums=[]
        for i in contenido:
            nums.append(float(i))
        ##################
        print(nums)
        self.numeros=nums
        self.sumar()
        self.text_lista_numeros.setText(str(self.numeros))

```